

BOOTCAMP: TRANSFORMACIÓN DIGITAL PARA LA DOCENCIA TÉCNICA

Nombre del participante:

Rosario Del Carmen Sorto Argueta

Institución:

Instituto Nacional de Osicala.

Tema: Script SQL con consultas CRUD y consultas JOIN funcionales sobre la base de datos existente.

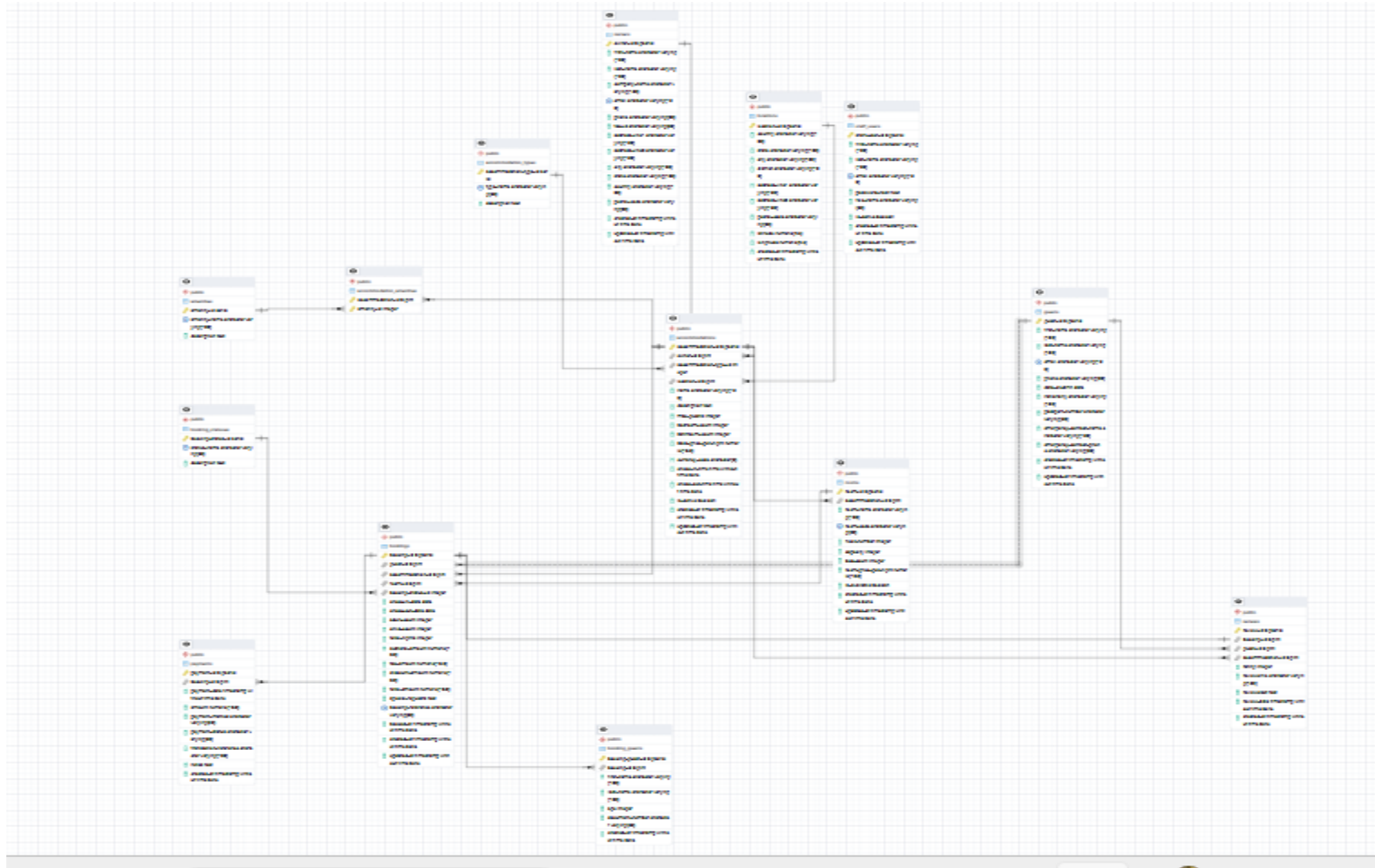
Link del Entregable:

<https://github.com/rosariosortoargueta/Tarea-base-de-datos->

Viernes 05 de junio de 2026

Esquema de la base de datos	3
Ejercicio 1: Selección básica de registros	4
Ejercicio 2: Proyección de columnas específicas	4
Ejercicio 3: Filtro numérico exacto (WHERE)	5
Ejercicio 4: Filtro de texto exacto (WHERE)	5
Ejercicio 5: Operadores de comparación condicional	6
Ejercicio 6: Ordenamiento de resultados (ORDER BY)	6
Ejercicio 7: Búsqueda de patrones en cadenas (LIKE)	7
Ejercicio 8: Límite de filas devueltas (LIMIT)	7
Ejercicio 9: Eliminación de duplicados (DISTINCT)	8
Ejercicio 10: Conteo total de registros (COUNT)	8
Ejercicio 11: Cálculo estadístico de promedio (AVG)	9
Ejercicio 12: Agrupación de datos por categorías (GROUP BY)	9
Ejercicio 13: Filtros sobre datos agrupados (HAVING)	10
Ejercicio 14: Unión interna de tablas (INNER JOIN)	10
Ejercicio 15:	11
Ejercicio 16:	11
Ejercicio 17: Accommodations con promedio de rating mayor a 3 (HAVING)	12
Ejercicio 18: Funcion de ventana: numero de fila para cada review por accommodation ..	12
Ejercicio 19: COALESCE: reemplazar NULLs en document_number de booking_guests	13
Ejercicio 20: COALESCE: reemplazar NULLs en phone de guests	13

Esquema de la base de datos



Ejercicio 1: Selección básica de registros

Query

```
-- 1. Buscar guests que contengan 'Mar' en el nombre (LIKE)
SELECT * FROM guests WHERE first_name ILIKE '%mar%';
```

Data Output

guest_id [PK] bigint	first_name character varying (100)	last_name character varying (100)	email character varying (150)	phone character varying (30)	date_of_birth date	nationality character varying (100)	passport_number character varying (50)	
1	3	María Teresa	López	stephanie15@example.c...	+34977338484	1952-09-18	Slowenien	[null]
2	10	Marcel	Lejeune	bianca28@example.com	427.086.6882	1961-10-23	Eritrea	[null]
3	11	Marc	Mateo	sharonhayden@example...	+33 3 58 63 85 46	1981-08-06	India	xu58771753
4	14	Ingmar	Macías	davisnicholas@example...	+33 (0)1 41 21 76 59	2006-03-05	Vereinigtes Königreich	[null]
5	47	Martín	Fonseca	bvilanova@example.net	922.690.3494x4665	1977-01-15	Tailandia	PM96910858
6	52	Margaud	Bates	silvocalvo@example.net	+49(0)4117 393483	1993-02-25	Gibraltar	aQ08737530
7	62	Marie	Vilar	cneuschaefer@example.c...	1-811-288-0133	2002-02-24	Costa Rica	[null]
8	94	Marcela	Figueroa	bertha79@example.net	1-069-804-2043x94070	1971-09-24	Suecia	uF10463681

Total rows: 8 Query complete 00:00:00.210 CRLF Ln 270, Col 1

Ejercicio 2: Proyección de columnas específicas

-- 2. Proyección de columnas: Muestra únicamente el nombre, apellido y correo electrónico.

SELECT first_name, last_name, email FROM staff_users;

Query

```
-- 2. Proyección de columnas: Muestra únicamente el nombre, apellido y correo electrónico.
SELECT first_name, last_name, email FROM staff_users;
```

Data Output

first_name character varying (100)	last_name character varying (100)	email character varying (150)	
1	Paulette	Maldonado	carstenmatthias@example.c...
2	Paul	Brunel	jgonzalez@example.com
3	Rita	Roche	xcrosby@example.org
4	Yolanda	Maillet	qkim@example.net
5	David	Maréchal	weihmannstella@example.org
6	Michael	Mireles	ocasillas@example.com
7	Roger	Scheel	epalacio@example.net
8	Ulrike	Vallée	gaetano71@example.org
9	Amelia	Grondin	camachocarlos@example.org
10	Margaret	Stanton	honore49@example.net

Total rows: 10 Query complete 00:00:00.276 CRLF Ln 278, Col 1

Ejercicio 3: Filtro numérico exacto (WHERE)

-- 3. Filtro exacto (WHERE): Encuentra todas las reseñas que tengan una calificación perfecta de 5.

```
SELECT * FROM reviews WHERE rating = 5;
```

The screenshot shows the PostgreSQL GUI with the query `SELECT * FROM reviews WHERE rating = 5;` executed. The results are displayed in a table with 16 rows. The columns are: review_id, booking_id, guest_id, accommodation_id, rating, review_title, and review_text. The rating column shows values of 5 for all rows.

review_id	booking_id	guest_id	accommodation_id	rating	review_title	review_text
1	5	98	46	15	5	Excellent service
2	13	14	49	12	5	Amazing stay!
3	17	37	91	17	5	Clean and cozy
4	18	15	29	11	5	Perfect for families
5	31	6	53	2	5	Perfect for families
6	32	81	1	3	5	Clean and cozy
7	33	27	82	10	5	Perfect for families
8	36	31	72	20	5	Hidden gem
9	39	87	46	10	5	Perfect for families
10	46	32	59	19	5	Great location

Ejercicio 4: Filtro de texto exacto (WHERE)

-- 4. Filtro de texto (WHERE): Busca al empleado o empleados cuyo apellido sea exactamente 'Maldonado'.

```
SELECT * FROM staff_users WHERE last_name = 'Maldonado';
```

The screenshot shows the PostgreSQL GUI with the query `SELECT * FROM staff_users WHERE last_name = 'Maldonado';` executed. The results are displayed in a table with 1 row. The columns are: staff_user_id, first_name, last_name, email, password_hash, and role. The last_name column shows the value 'Maldonado'.

staff_user_id	first_name	last_name	email	password_hash	role
1	Paulette	Maldonado	carstenmatthias@example.c...	\$2b\$12\$IK6SP3H2AagxddqW8QjdWmHwCMPInOyFXRQcxzWQ7dUm11BnNR...	Ma

Ejercicio 5: Operadores de comparación condicional

-- 5. Operadores de comparación: Muestra las reseñas con calificación menor o igual a 2 (útil para quejas).

```
SELECT * FROM reviews WHERE rating <= 2;
```

The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL code:

```
-- 5. Operadores de comparación: Muestra las reseñas con calificación menor o igual a 2 (útil para quejas).
SELECT * FROM reviews WHERE rating <= 2;
```

The Data Output tab displays the results of the query, showing 10 rows of review data where the rating is 2 or less. The columns are: review_id, booking_id, guest_id, accommodation_id, rating, review_title, and review_text.

review_id	booking_id	guest_id	accommodation_id	rating	review_title	review_text
1	1	39	60	2	Great location	Pour plaindre regretter devoir rentrer doux.
2	3	85	55	4	Amazing stay!	Esfuerzo ocho estado ahora. Principales fácil es organización resulta de esta.
3	4	13	2	5	Very comfortable	Précipiter tandis que moyen lui.
4	9	78	54	6	Great location	Les cuestión tratamiento edad viene temas.
5	10	90	90	2	Would visit again	Llama todo trabajo tomar aquellos. Muy habrá momento grupo está falta cuadro es
6	12	99	87	15	Clean and cozy	Vif muerd chaleur second. Réduire note espoir casser chant.
7	19	88	89	20	Not as described	Citizen ahead method audience. Return or for cup quality.
8	21	26	85	13	Perfect for families	Intérieur joue succès chasse.
9	22	92	79	2	Great location	Hiife dein Schüler Oma. Erde darin schlimm Baum dein.
10	25	95	43	9	Perfect for families	Safe mean international hour special recent better.

Successfully run. Total query runtime: 172 msec. 22 rows affected.

Ejercicio 6: Ordenamiento de resultados (ORDER BY)

-- 6. Ordenamiento (ORDER BY): Lista todas las reseñas ordenadas por su calificación de mayor a menor.

```
SELECT * FROM reviews ORDER BY rating DESC;
```

The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL code:

```
-- 5. Operadores de comparación: Muestra las reseñas con calificación menor o igual a 2 (útil para quejas).
SELECT * FROM reviews WHERE rating <= 2;

-- 6. Ordenamiento (ORDER BY): Lista todas las reseñas ordenadas por su calificación de mayor a menor.
SELECT * FROM reviews ORDER BY rating DESC;
```

The Data Output tab displays the results of the query, showing 10 rows of review data sorted by rating in descending order. The columns are: review_id, booking_id, guest_id, accommodation_id, rating, review_title, and review_text.

review_id	booking_id	guest_id	accommodation_id	rating	review_title	review_text
1	31	6	53	2	Perfect for families	M mujeres análisis pueblo parte internacional.
2	32	81	1	3	Clean and cozy	Satisfaire intérêt manquer salle surtout humide fait. Consulter sourire bataille foi.
3	33	27	82	10	Perfect for families	Hacer muchas ve necesidad.
4	59	19	91	6	Perfect for families	Bien mirada serán varias vivir buscar personas. En baja llama toma ocasión hacía c
5	36	31	72	20	Hidden gem	Expect what peace claim movement consider.
6	51	5	85	20	Very comfortable	Cine distintos por dólares policia gonzález buena.
7	13	14	49	12	Amazing stay!	Midi haut ton appartenir bois séparer.
8	55	17	18	14	Perfect for families	Situación máximo medios serán.
9	17	37	91	17	Clean and cozy	Television would medical less phone decade. Create someone rate.
10	18	15	29	11	Perfect for families	Circumstance toile art facile. Habiter espèce révolution rayon apprendre.

Successfully run. Total query runtime: 224 msec. 60 rows affected.

Ejercicio 7: Búsqueda de patrones en cadenas (LIKE)

-- 7. Búsqueda de patrones (LIKE): Encuentra los correos de empleados que terminen en '@example.com'.

```
SELECT first_name, email FROM staff_users WHERE email LIKE '%@example.com';
```

The screenshot shows a PostgreSQL IDE interface. The left sidebar displays a tree view of the database structure, including 'Databases (5)' and 'public' schema. The main query editor contains the following SQL code:

```
-- 7. Búsqueda de patrones (LIKE): Encuentra los correos de empleados que terminen en '@example.com'.
SELECT first_name, email FROM staff_users WHERE email LIKE '%@example.com';
```

The 'Data Output' tab is active, showing the results of the query in a table with 3 rows:

first_name	email
Paulette	carstenmatthias@example.c...
Paul	jgonzalez@example.com
Michael	ocasillas@example.com

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.191'. A green message box at the bottom right states 'Successfully run. Total query runtime: 191 msec. 3 rows affected.'

Ejercicio 8: Límite de filas devueltas (LIMIT)

-- 8. Límite de resultados (LIMIT): Muestra solo los primeros 5 empleados registrados en el sistema.

```
SELECT * FROM staff_users ORDER BY staff_user_id ASC LIMIT 5;
```

The screenshot shows a PostgreSQL IDE interface. The left sidebar displays a tree view of the database structure. The main query editor contains the following SQL code:

```
-- 8. Límite de resultados (LIMIT): Muestra solo los primeros 5 empleados registrados en el sistema.
SELECT * FROM staff_users ORDER BY staff_user_id ASC LIMIT 5;
```

The 'Data Output' tab is active, showing the results of the query in a table with 5 rows:

staff_user_id	first_name	last_name	email	password_hash	role_name
1	Paulette	Maldonado	carstenmatthias@example.c...	\$2b\$12\$IK6SP3H2AagxddqW8QjdWmHwCmPinOyFXRQcxzWQ7dUm11...	Manager
2	Paul	Brunel	jgonzalez@example.com	\$2b\$12\$7slQikFxAlikPusSjud5MPEJwWtniefZhfnnuwnUG9Z8aPB4MM7...	Admin
3	Rita	Roche	xrosby@example.org	\$2b\$12\$Hr4YxXbc3IF6k7HaePV1AgHR8W06UjCufdXUJoqyNABPHNMUK...	Receptionist
4	Yolanda	Maillet	qkim@example.net	\$2b\$12\$5r5xYbjnUX0sNs27107a2icJyHfntTn40ilMphPJduU9iNePpmJ6w...	Receptionist
5	David	Maréchal	weihmannstella@example.org	\$2b\$12\$3tBMsKJwtaApnTB0IT2CwsikL01IHq2eebFbKzK4lH9DWwnVTs...	Accountant

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.197'. A green message box at the bottom right states 'Successfully run. Total query runtime: 197 msec. 5 rows affected.'

Ejercicio 9: Eliminación de duplicados (DISTINCT)

-- 9. Buscar accommodations que contengan 'Villa' (LIKE)

The screenshot shows the PostgreSQL GUI interface. The left sidebar displays the database structure, including the 'public' schema. The main query editor contains the following SQL code:

```
298 SELECT * FROM staff_users ORDER BY staff_user_id ASC LIMIT 5;  
299  
300 -- 9. Buscar accommodations que contengan 'Villa' (LIKE)  
301 SELECT * FROM accommodations WHERE name ILIKE '%Rustic Lodge Ville%';  
302  
303
```

The 'Data Output' tab at the bottom shows the results of the query. The table has the following columns: accommodation_id, owner_id, accommodation_type_id, location_id, name, description, max_guests, bedroom_count, and bathroom_count. The first row of data is:

accommodation_id	owner_id	accommodation_type_id	location_id	name	description	max_guests	bedroom_count	bathroom_count
1	1	9	7	19	Rustic Lodge Ville	Asimismo mes principios peso jov...	4	2

Ejercicio 10: Conteo total de registros (COUNT)

-- 10. Conteo total (COUNT): Cuenta cuántas reseñas en total existen registradas en la tabla.

```
SELECT COUNT(*) AS total_reviews FROM reviews;
```

The screenshot shows the PostgreSQL GUI interface. The left sidebar displays the database structure, including the 'public' schema. The main query editor contains the following SQL code:

```
303 -- 10. Filtros múltiples (AND): Busca reseñas con calificación de 4 que pertenezcan al alojamiento con ID 2.  
304 SELECT * FROM reviews WHERE rating = 4 AND accommodation_id = 2;  
305  
306 -- 10. Conteo total (COUNT): Cuenta cuántas reseñas en total existen registradas en la tabla.  
307 SELECT COUNT(*) AS total_reviews FROM reviews;  
308
```

The 'Data Output' tab at the bottom shows the results of the query. The table has the following columns: total_reviews. The first row of data is:

total_reviews
60

Ejercicio 11: Cálculo estadístico de promedio (AVG)

-- 12. Promedio (AVG): Calcula la calificación promedio de todas las reseñas de la plataforma.

```
SELECT AVG(rating) AS average_rating FROM reviews;
```

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure, including the 'accommodation' database and its tables. The main window shows the 'Query' tab with the following SQL code:

```
-- 10. Conteo total (COUNT): Cuenta cuántas reseñas en total existen registradas en la tabla.
SELECT COUNT(*) AS total_reviews FROM reviews;

-- 12. Promedio (AVG): Calcula la calificación promedio de todas las reseñas de la plataforma.
SELECT AVG(rating) AS average_rating FROM reviews;
```

The 'Data Output' tab shows the result of the query:

average_rating
3.1500000000000000

The status bar at the bottom indicates: 'Total rows: 1 Query complete 00:00:00.167'. A green message box at the bottom right states: 'Successfully run. Total query runtime: 167 msec. 1 rows affected.'

Ejercicio 12: Agrupación de datos por categorías (GROUP BY)

-- 12. Agrupación (GROUP BY): Muestra cuántas reseñas ha recibido cada alojamiento de forma individual. SELECT accommodation_id, COUNT(review_id) AS total_reviews FROM reviews GROUP BY accommodation_id;

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure. The main window shows the 'Query' tab with the following SQL code:

```
-- 12. Agrupación (GROUP BY): Muestra cuántas reseñas ha recibido cada alojamiento de forma individual.
SELECT accommodation_id, COUNT(review_id) AS total_reviews
FROM reviews
GROUP BY accommodation_id;
```

The 'Data Output' tab shows the result of the query:

accommodation_id	total_reviews
1	8
2	11
3	19
4	4
5	14
6	3
7	17
8	20
9	7
10	13
11	9

The status bar at the bottom indicates: 'Total rows: 20 Query complete 00:00:00.210'. The bottom right corner shows the system clock at 21:16.

Ejercicio 13: Filtros sobre datos agrupados (HAVING)

-- 13. Agrupación con filtro (HAVING): Filtra la agrupación anterior para mostrar solo alojamientos con más de 5 reseñas. `SELECT accommodation_id, COUNT(review_id) AS total_reviews FROM reviews GROUP BY accommodation_id HAVING COUNT(review_id) > 5;`

The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL code:

```
-- 13. Agrupación con filtro (HAVING): Filtra la agrupación anterior para mostrar solo alojamientos con más de 5 reseñas.
SELECT accommodation_id, COUNT(review_id) AS total_reviews
FROM reviews
GROUP BY accommodation_id
HAVING COUNT(review_id) > 5;
```

The query has been executed successfully. The Data Output pane shows the following results:

accommodation_id	total_reviews
1	8
2	11
3	20
4	10
5	5
6	2
7	15

A status message at the bottom indicates: "Successfully run. Total query runtime: 208 msec. 7 rows affected."

Ejercicio 14: Unión interna de tablas (INNER JOIN)

Agrupación por calificación: Cuenta cuántas reseñas existen para cada nivel de calificación (1 al 5) y las ordena. `SELECT rating, COUNT(*) AS count_of_rating FROM reviews GROUP BY rating ORDER BY rating DESC;`

The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL code:

```
WHERE accommodation_id = 4;

-- 14 Agrupación por calificación: Cuenta cuántas reseñas existen para cada nivel de calificación (1 al 5) y las ordena.
SELECT rating, COUNT(*) AS count_of_rating
FROM reviews
GROUP BY rating
ORDER BY rating DESC;
```

The query has been executed successfully. The Data Output pane shows the following results:

rating	count_of_rating
5	16
4	14
3	8
2	7
1	15

Ejercicio 15:

Subconsulta escalar: Muestra las reseñas cuya calificación sea estrictamente mayor al promedio general.

The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL code:

```
-- 15. Subconsulta escalar: Muestra las reseñas cuya calificación sea estrictamente mayor al promedio general.
SELECT review_title, rating FROM reviews WHERE rating > (SELECT AVG(rating) FROM reviews);
```

The Data Output pane shows the results of the query:

review_title	rating
Excellent service	5
Not as described	4
Hidden gem	4
Clean and cozy	4
Amazing stay!	5
Hidden gem	4
Amazing stay!	4
Clean and cozy	5
Perfect for families	5
Very comfortable	4
Great location	4

Total rows: 30 Query complete 00:00:00.172

Ejercicio 16:

-- 16. Lógica condicional (CASE): Clasifica las reseñas en 'Positiva', 'Neutral' o 'Negativa' según su puntuación.

The screenshot shows the PostgreSQL IDE interface. The query editor contains the following SQL code:

```
-- 16. Lógica condicional (CASE): Clasifica las reseñas en 'Positiva', 'Neutral' o 'Negativa' según su puntuación.
SELECT review_id, rating,
CASE
  WHEN rating >= 4 THEN 'Positiva'
  WHEN rating = 3 THEN 'Neutral'
  ELSE 'Negativa'
END AS sentimiento_resena
FROM reviews;
```

The Data Output pane shows the results of the query:

review_id	rating	sentimiento_resena
1	1	Negativa
2	2	Neutral
3	3	Negativa
4	4	Negativa
5	5	Positiva
6	6	Neutral
7	7	Positiva
8	8	Positiva

Total rows: 60 Query complete 00:00:00.191

Ejercicio 17: Accommodations con promedio de rating mayor a 3 (HAVING)

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure, including the 'public' schema. The main query editor contains the following SQL code:

```
405 -- 17. Accommodations con promedio de rating mayor a 3 (HAVING)
406
407
408
409 SELECT a.name, ROUND(AVG(r.rating), 2) AS promedio
410 FROM accommodations a
411 INNER JOIN reviews r ON a.accommodation_id = r.accommodation_id
412 GROUP BY a.accommodation_id, a.name
413 HAVING AVG(r.rating) > 3;
414
```

The 'Data Output' tab shows the results of the query:

	name	promedio
1	Boutique Suite los bajos	3.33
2	Luxury Escape de la Montaña	4.67
3	Boutique Retreat Ville	4.00
4	Luxury Getaway los altos	5.00
5	Rustic Retreat furt	3.25
6	Rustic Lodge boeuf	4.00
7	Charming Stay los bajos	3.50
8	Boutique Lodge los bajos	3.50

Total rows: 10 Query complete 00:00:00.177

Ejercicio 18: Funcion de ventana: numero de fila para cada review por accommodation

The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure. The main query editor contains the following SQL code:

```
524
525 -- 18 Funcion de ventana: numero de fila para cada review por accommodation
526
527
528 SELECT
529     review_id,
530     accommodation_id,
531     guest_id,
532     rating,
533     ROW_NUMBER() OVER (PARTITION BY accommodation_id ORDER BY review_id) AS review_num
534 FROM reviews;
```

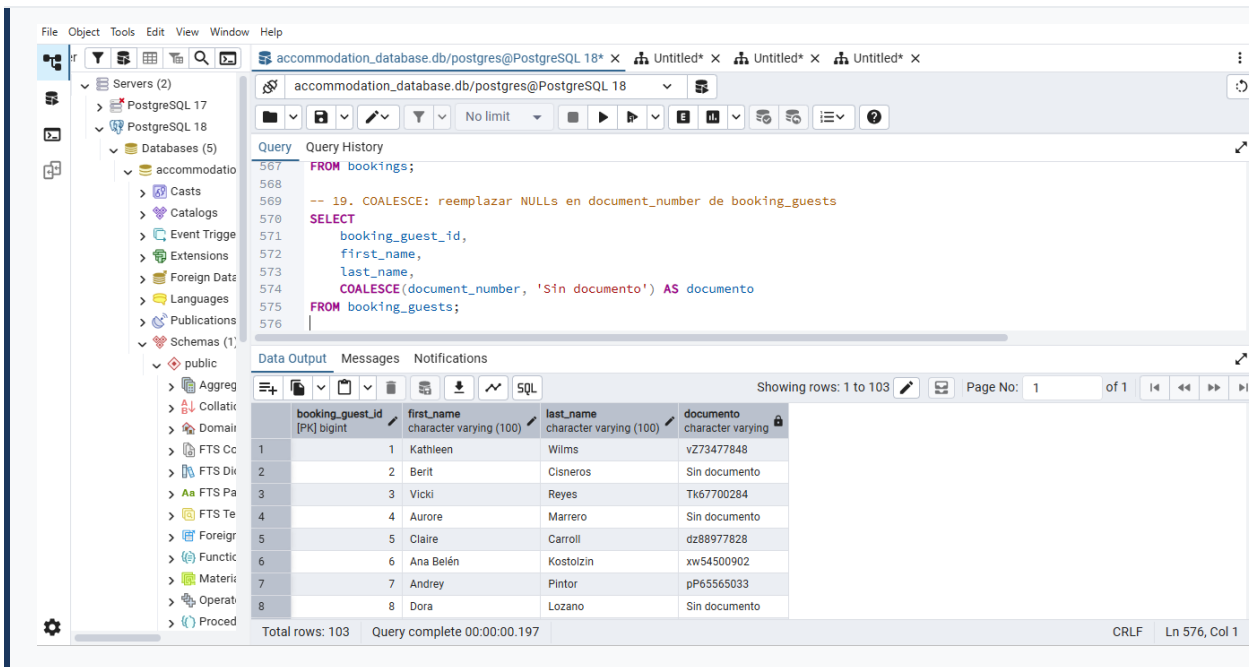
The 'Data Output' tab shows the results of the query:

	review_id [PK] bigint	accommodation_id bigint	guest_id bigint	rating integer	review_num bigint
1	30	1	96	1	1
2	47	1	64	5	2
3	1	2	60	1	1
4	10	2	90	2	2
5	22	2	79	1	3
6	31	2	53	5	4
7	6	3	94	3	1
8	27	3	31	4	2

Total rows: 60 Query complete 00:00:00.185

Successfully run. Total query runtime: 185 msec. 60 rows affected.

Ejercicio 19: COALESCE: reemplazar NULLs en document_number de booking_guests



The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure, including the 'public' schema. The main query editor contains the following SQL code:

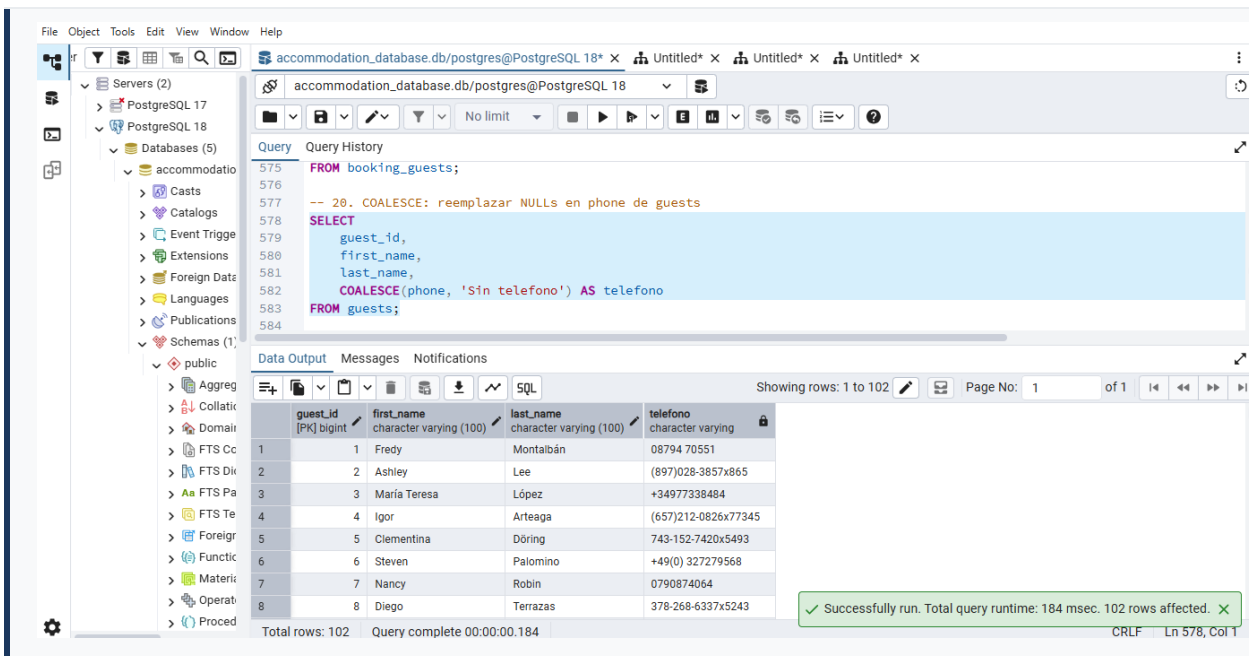
```
567 FROM bookings;
568
569 -- 19. COALESCE: reemplazar NULLs en document_number de booking_guests
570 SELECT
571     booking_guest_id,
572     first_name,
573     last_name,
574     COALESCE(document_number, 'Sin documento') AS documento
575 FROM booking_guests;
576
```

The 'Data Output' tab shows the results of the query, displaying 103 rows. The columns are: booking_guest_id (PK) bigint, first_name character varying (100), last_name character varying (100), and documento character varying. The data is as follows:

booking_guest_id	first_name	last_name	documento
1	Kathleen	Wilms	vZ73477848
2	Berit	Cisneros	Sin documento
3	Vicki	Reyes	Tk67700284
4	Aurore	Marrero	Sin documento
5	Claire	Carroll	dz88977828
6	Ana Belén	Kostolzin	xw54500902
7	Andrey	Pintor	pP65565033
8	Dora	Lozano	Sin documento

Total rows: 103. Query complete 00:00:00.197.

Ejercicio 20: COALESCE: reemplazar NULLs en phone de guests



The screenshot shows the PostgreSQL IDE interface. The left sidebar displays the database structure, including the 'public' schema. The main query editor contains the following SQL code:

```
575 FROM booking_guests;
576
577 -- 20. COALESCE: reemplazar NULLs en phone de guests
578 SELECT
579     guest_id,
580     first_name,
581     last_name,
582     COALESCE(phone, 'Sin telefono') AS telefono
583 FROM guests;
584
```

The 'Data Output' tab shows the results of the query, displaying 102 rows. The columns are: guest_id (PK) bigint, first_name character varying (100), last_name character varying (100), and telefono character varying. The data is as follows:

guest_id	first_name	last_name	telefono
1	Fredy	Montalbán	08794 70551
2	Ashley	Lee	(897)028-3857x865
3	María Teresa	López	+34977338484
4	Igor	Arteaga	(657)212-0826x77345
5	Clementina	Döring	743-152-7420x5493
6	Steven	Palomino	+49(0) 327279568
7	Nancy	Robin	0790874064
8	Diego	Terrazas	378-268-6337x5243

Total rows: 102. Query complete 00:00:00.184. Successfully run. Total query runtime: 184 msec. 102 rows affected.